



DLDBSOOFPP01

OBJECT ORIENTATED AND FUNCTIONAL PROGRAMMING WITH PYTHON

HABIT TRACKER : FINALIZATION PHASE

Name and Surname	Matriculation Number
Dylan Vosloo	9212174

March 24, 2025

1 Discussion of Solution

1.1 Content and Context

The Habit Tracker aims to allow a user to accurately define and track their habits with the aim of being more consistent with completing tasks.

The Tracker allows a user to add and remove habits, display information about all habits or habits within a particular period, either daily or weekly. Further, the user can analyse their habits by defining which habit has the highest "streak". A streak is the number of consecutive successful completions of the habit within the designated period. This information can be displayed by period or overall.

In order to achieve this, the application is coded in Python 3.11 and makes use of a class based and functional programming approach.

An overall Habit class is created in order to work with all habitual information. This includes the Habit name, start date, complete date, a completion variable (has it been completed in this time period or not), the period of the habit, the habit number and the streak number. Each habit is instantiated within the Habit class, thus a instance of a habit is created. This allows for repetitive code as every habit can be referenced by the same variables, it is only the instance of habit that changes.

Along with the class approach, modular and functional programming is incorporated into the tracker. This allows certain operations like "display" or "create habit" to be grouped into a single procedure or function which can pass information through parameters into the module. This is helpful for when the user needs to input information, which would differ each time.

In order to store the information permanently and update it accordingly, a SQL approach is used. This makes use of the 'sqlite3' Python library. This database links with the class definition of a habit. When creating a habit, an instance of the habit class is passed to the database. When working with any information from the database, the information is again instantiated as an instance of the Habit class.

The database is populated with pre-recorded habit data that the user is allowed to use to test the application. This information is automatically added to the 'Habits.db' when the program runs for the first time and a database is not detected. The user can remove the data through the "delete-habit" command and the information will not be rewritten to the database on the next application sartup.

Multiple Python files are used in order to group sections of the project in a logical manner. These files include a 'main.py' which includes the class definition, a 'Habits.db' which has everything to do with adding/removing information to the database as well as reading information from the database. A 'Click.py' which incorporates a CLI (Command Line Interface) for the user to interact. This is technically the main file as it starts the main processes of the program.

The CLI, implemented through the Click Library, has a useful "help" function which allows the user to navigate through the application and actively search what each function does and how to use it. This greatly assists the user with knowing what to do and how to navigate the application. However, if the user is to make mistakes when working with the programme, many instances of validation and verification have been implemented.

Whenever the user types in an input for adding/removing a habit they are asked to confirm the input allowing for an extra check against small errors or typos. Finally, a 'Test.py' file is also included to allow the user to test the program. Certain functions of the tracker are run and due to their being pre-defined data, a specific outcome is expected. This unit test can be run in order to check that all aspects of the program are functional.

2 Experience of Implementation

The project served as a challenge as many concepts needed to be understood in order to achieve the desired outcomes. The initial struggle is to get everything "talking" to each other. Concepts such as reading information into/from a database and what format does it take? How does it interact with the class? Initially this served as an issue, but eventually a solution was discovered. Once this concept worked it was possible to expand rapidly with other similar functions. The next hurdle was the CLI as this also needed to be incorporated and the most challenging aspect is incorporating everything. This meant some adjustments to code already in place, which took time, but again once working it is straight forward to implement for the rest of the functions. Finally, compiling the program into a .exe proved a challenge as many DLL files seemed to be overlooked by the compiler. However, the final completed project is worth the price of admission.

Other aspects such as determining if a habit has been completed on time required some critical thinking and served as a fun challenge to implement.

3 Notable Features

The addition of the database instead of a JSON file felt like a better quality implementation, however there were some challenges in its implementation that, when overcome, left a triumphant feeling.

The application has a self-help feature where the user can work through the program themselves with relative ease and be able to learn how the application works and how to use it. This feels like a notable feature as it makes the application more autonomous. Further, the tracker includes a .exe file which can stand alone and the user may run without prior knowledge of Python.

The code does support validation and verification checks that allow the user to avoid any small errors or mistakes when inputting information into the tracker. This can go a long way as the user no longer needs to delete an incorrectly named habit. Suitable prompts and messages have also been added to clarify what mistakes have been made and to make the usage of the application smooth.

4 Github Repository Link

https://github.com/Vosssie/Cloud_Programming.git